Section 10 : Characters and Strings

Information

- Title: `Section 10 : Characters and Strings`
- Authors: `ETIENNE KOA`

Section Overview

- Characters and Strings
  - Character functions
  - `C-style` Strings
  - Workings with `C-style` Strings
  - `C++` Strings
  - Working with `C++` Strings

Characters Functions

```
#include <cctype>
```

```
#include <cctype>
function_name(char)
```

- Fonction pour tester les caractères
- Fonctions de conversion de la casse des caractères.

- Tester les cractères

`isalpha(c)`	True if c is a letter
`isalnum(c)`	True if c is a letter or digit
`isdigit(c)`	True if c is a digit
`islower(c)`	True if c is lowercase letter
`isprint(c)`	True if c is a printable character
`ispunct(c)`	True if c is a punctuation character
`isupper(c)`	True if c is an uppercase letter
`isspace(c)`	True if c is whitespace

Figure 1: characters-functions

- Conversion de caractères

`tolower(c)`	returns lowercase of c
`toupper(c)`	returns uppercase of c

Figure 2: characters-functions

Chaînes de style `C`

- Séquence de caractères
  - contigu en mémoire
  - implémenté sous forme de tableau de caractères
  - terminé par un caractère nul (null)
    - `null` - caractère avec une valeur de zéro
  - Appelé chaînes terminées par zéro ou null.
- Chaîne littérale
  - Séquence de caractères entre guillemets doubles, par ex. “Franck”
  - constante
  - terminé par un caractère nul

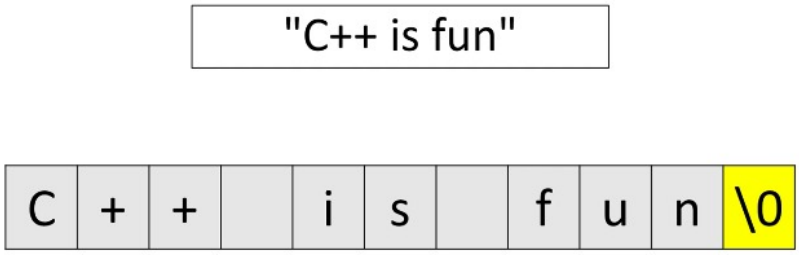

Figure 3: c-style

- déclarer des variables

```
char my_name[] {"Frank"};
```

F	r	a	n	k	\0

```
my_name[5] = 'y'; // Problem
```

```
char my_name[8] {"Frank"};
```

F	r	a	n	k	\0	\0	\0

```
my_name[5] = 'y'; // OK
```

```
char my_name[8];
```

?	?	?	?	?	?	?	?

```
my_name = "Frank";          // Error

strcpy(my_name, "Frank"); // OK
```

`#include <cstring>`

- Fonctions qui fonctionnent avec les chaînes « C-style»
  - Copie
  - Concaténation
  - Comparaison
  - Recherche
  - et autres

- Quelques exemples

```
char str[80];

strcpy(str, "Hello "); // copy

strcat(str, "there");  // concatenate

cout << strlen(str);   // 11

strcmp(str, "Another"); // > 0
```

Figure 4: c-style

`#include <cstdlib>`

- Fonctions à usage général
- Inclut des fonctions pour convertir les chaînes de style C en
  - integer
  - float
  - long
  - etc

`C++` Strings

- `std::string` est une classe dans la `Standard Template Library`.
  - `#include <cstring>`
  - Espace de noms `std`
  - contigu en mémoire
  - taille dynamique
  - travailler avec les flux d'entrée et de sortie
  - de nombreuses fonctions membres utiles
  - nos opérateurs familiers peuvent être utilisés `(+, =, <, <=, >, >=, +=, ==, !=, [] ...)`.
  - peut être facilement converti en « style C » si nécessaire.
  - plus sûr
- Déclaration et initialisation
- Assigner `=`

```
#include <string>
using namespace std;

string s1;                  // Empty
string s2 {"Frank"};        // Frank
string s3 {s2};             // Frank
string s4 {"Frank", 3};     // Fra
string s5 {s3, 0, 2};       // Fr
string s6 (3, 'X');         // XXX
```

Figure 5: c-style

```
string s1;
s1 = "C++ Rocks!";

string s2 {"Hello"};
s2 = s1;
```

- Concaténation

```
string part1 {"C++"};
string part2 {"is a powerful"};

string sentence;

sentence = part1 + " " + part2 + " language";
        // C++ is a powerful language

sentence = "C++" + " is powerful"; // Illegal
```

Figure 6: c-style

- Accès aux méthodes `characters[]` et `at()`

```
string s1;
string s2 {"Frank"};

cout << s2[0] << endl;    // F
cout << s2.at(0) << endl; // F

s2[0] = 'f';      // frank
s2.at(0) = 'p';   // prank

string s1 {"Frank"};

for (int c: s1)
     cout << c << endl;

70   // F
114  // r
97   // a
110  // n
107  // k
0    // null character
```

- Comparaison

  `== != > >= < <=`

- Les objets sont comparés caractère par caractère lexicalement.
  - Peut comparer :

    * deux objets `std::string`
    * Objet `std::string` et littéral de chaîne `C-style`.
    * Objet `std::string` et variable de chaîne `C-style`.

```
string s1 {"Apple"};
string s2 {"Banana"};
string s3 {"Kiwi"};
string s4 {"apple"};
string s5 {s1};            // Apple

s1 == s5            // True
s1 == s2            // False
s1 != s2            // True
s1 < s2             // True
s2 > s1             // True
s4 < s5             // False
s1 == "Apple";      // True
```

Figure 7: c-style

- Sous-chaîne - `substr()`
  - Extraire une sous-chaîne d'un `std::string`

    `objet.substr(start_index, length)`
- Recherche - `find()`
  - Renvoie l'index d'une sous-chaîne dans un `std::string`

    ```
    objet.find (search_string)
    ```
- Suppression de caractères - `erase()` et `clear()`
  - Supprime une sous-chaîne de caractères d'un `std::string`

    `object.erase(start_index,n length)`
- Autres méthodes utiles

```
string s1 {"This is a test"};

cout << s1.substr(0,4); // This
cout << s1.substr(5,2); // is
cout << s1.substr(10,4);   // test
```

Figure 8: c-style

```
string s1 {"This is a test"};

cout << s1.find("This");  // 0
cout << s1.find("is");    // 2
cout << s1.find("test");  // 10
cout << s1.find('e');     // 11
cout << s1.find("is", 4);     // 5
cout << s1.find("XX");    // string::npos
```

Figure 9: c-style

```
string s1 {"This is a test"};

cout << s1.erase(0,5); // is a test
cout << s1.erase(5,4); // is a
s1.clear();              // empties string s1
```

Figure 10: c-style

```
string s1 {"Frank"};

cout << s1.length() << endl;  // 5

s1 += " James";
cout << s1 << endl;        // Frank James
```

Figure 11: c-style

- Bien d'autres encore...
- Saisissez `>>` et `getline()`
  - Lecture de `std::string` depuis `cin`

```
string s1;
cin >> s1;          // Hello there
                    // Only accepts up to the first space
cout << s1 << endl;     // Hello

getline(cin, s1);  // Read entire line until \n
cout << s1 << end;      // Hello there

getline(cin, s1, 'x');  // this isx
cout << s1 << endl;     // this is
```

Figure 12: c-style